VIGÍA SST

Plan de gestión de la configuración del software

Proyecto: Vigía SST. Versión documental: 0.2. Fecha de preparación: 18 de octubre de 2026. Estado: propuesta inicial para revisión académica.

1. Propósito y alcance

Identificar y controlar los elementos que forman el prototipo, mantener trazabilidad de cambios y permitir reproducir una versión. Incluye código, dependencias, pruebas, documentación, variables de entorno de ejemplo y configuración de despliegue.

2. Elementos de configuración

Elemento	Ubicación	Control
API y permisos	server.js, lib/	Git y pruebas de integración
Interfaz	public/	Git y revisión visual
Dependencias	package.json, pnpm-lock.yaml	Versiones registradas y lockfile
Pruebas y automatización	test/, .github/workflows/	GitHub Actions
Documentación	README.md, docs/	Revisión con cada cambio
Despliegue	render.yaml	Configuración versionada
Variables de ejemplo	.env.example	Sin credenciales reales

Los datos de usuarios, sesiones, reportes y adjuntos se almacenan en MongoDB y no se versionan en Git. .env, node_modules/ y data/ quedan excluidos.

3. Repositorio y líneas base

Repositorio: https://github.com/HEFRA-ws/ProyectoIngSoftware. La rama main contiene la versión revisada disponible. La línea base inicial se identifica con la etiqueta v0.1.0; la presente revisión corresponde a la versión 0.2. Una etiqueta identifica código, no certifica despliegue ni cumplimiento de todos los requisitos.

4. Control de cambios

Crear una issue con problema, criterio de aceptación y evidencia. Para el trabajo posterior, usar ramas feature/descripcion, fix/descripcion o docs/descripcion. Abrir pull request con descripción, validación y limitaciones. Integrar cuando las comprobaciones hayan pasado y el cambio se haya revisado. El primer prototipo puede cargarse directamente en main; no se simulan revisiones entre integrantes.

Usar commits con propósito concreto: feat:, fix:, test: y docs:. La autoría debe reflejar la cuenta y la asistencia utilizadas; no se inventa participación del equipo.

5. Construcción y reproducibilidad

Node.js 24 y pnpm. Instalar con pnpm install --frozen-lockfile --ignore-scripts. Ejecutar pnpm test y pnpm start. El lockfile se modifica únicamente cuando cambian dependencias. El MongoDB local descarga un binario en la primera ejecución; en producción se utiliza MongoDB Atlas.

6. Ambientes y secretos

Desarrollo: MongoDB local persistente o base Atlas independiente. Pruebas: MongoDB aislado temporal. Despliegue: Render y Atlas, pendientes de configurar y verificar. Utilizar bases distintas para pruebas y demostración. Definir MONGODB_URI y DEMO_PASSWORD como secretos de ejecución; no registrarlos en issues ni commits.

7. Verificación de configuración

Antes de publicar: comprobar git status, revisar los archivos incluidos, ejecutar pruebas, verificar la interfaz y actualizar documentación. GitHub Actions comprueba sintaxis y pruebas de integración. Confirmar el resultado del workflow en GitHub; la existencia del archivo YAML no acredita una ejecución exitosa.

8. Respaldo y recuperación

GitHub conserva el historial del código. Para datos, definir y ensayar una exportación de MongoDB antes de usar información que se deba conservar. En desarrollo, el directorio data/ contiene archivos WiredTiger; detener MongoDB antes de realizar una copia en frío. En Atlas verificar las opciones de respaldo disponibles para el plan seleccionado. El respaldo de datos en cloud aún no está implementado.

Para revertir código publicado, preferir git revert al borrado o reescritura del historial compartido. Una reversión de código no revierte automáticamente los datos; evaluar compatibilidad del modelo antes de desplegar.

9. Responsabilidades

Coordinación: priorizar cambios y autorizar la línea base académica. Desarrollo: implementar y documentar. QA: reproducir defectos y validar correcciones. Despliegue: configurar ambiente y verificar persistencia. Las personas responsables se asignarán con el equipo; no se presenta una aprobación ficticia.

10. Migración y compatibilidad

La versión 0.2 separa events de accidentReports. Al arrancar se renombra la colección antigua reports y se cambia reportId a eventId en evidencia. Se conservan códigos e historial; si existen reports y events simultáneamente el arranque se detiene para revisión. No se generan informes ni gravedad retrospectiva. Actualizar interfaz y servidor juntos, pues la API utiliza /api/events. Antes de migrar información que se deba conservar, preparar un respaldo de la base.

11. Registro del estado

El plan de sprints, las issues y el historial Git registran trabajo completado y pendiente. La memoria personal de estudio y las credenciales no forman parte del repositorio público.